GOVERNMENT ARTS & SCIENCE COLLEGE

TITTAGUDI

REGISTER NO	: 24 131191802521007
NAME	: CHANDHIRAKASU P
NM ID	: 9CB33F1F98FAF0ABB3D462B206F698BF
DEPARTMENT NAME	: B.Sc.Computer Science
YEAR	: III - YEAR
GROUP PROJECT NAME	: **CRICKET PLAYER PERFORMANCE TRACKING APPLICATION : CRICKETATS**

evidence in the later pages.

Table of Contents

This 25-page supplement is designed to be appended to a larger project report.

1. Project overview and documentation purpose
2. Problem statement, objectives and scope
3. System architecture model
4. Context diagram
5. DFD Level 0
6. DFD Level 1
7. Use case model
8. Activity model
9. Sequence model - user login
10. Sequence model - performance entry
11. Entity relationship model
12. Class model
13. Database schema model
14. Player data model
15. Match performance model
16. Performance metric model
17. Role and access model
18. UI/navigation model
19. App Manager implementation evidence
20. Player object field model evidence
21. Custom tab model evidence
22. Validation, business rules and testing
23. Deployment and maintenance
24. Conclusion and future enhancement

1. Project Overview and Documentation Purpose

CRICKTATs is a cricket player performance tracking application intended to organize player profiles, match information and performance records in a single system. The model section translates the application into clear software-engineering views so that the implementation can be explained in a project review, viva or technical report.

The screenshots supplied with the project indicate a Salesforce Lightning-based configuration approach, including an App Manager entry, a custom Player object with fields and relationships, and custom object tabs. The diagrams in this supplement describe the logical behavior of the application rather than claiming implementation details that are not visible in the supplied evidence.

Documentation objectives

- Provide a consistent set of analysis and design models for the application.
- Explain how player, match and performance information flows through the system.
- Connect the logical models with the supplied implementation screenshots.
- Provide enough structured material to be inserted into a larger academic project report.

Primary modules: Player Management, Team Management, Match Management, Performance Entry, Statistics and Reports, User Access, and Administration.

2. Problem Statement, Objectives and Scope

Problem statement

Cricket performance information is often collected across score sheets, spreadsheets or isolated records. Such fragmentation makes it harder to maintain a consistent player history, retrieve match-by-match results and compare performance trends. A centralized application can reduce duplicate entry and provide a structured view of the available statistics.

Objectives

- Create and maintain player profiles with searchable attributes.
- Store match-level information and associate performances with the correct player and match.
- Capture key batting, bowling and fielding indicators.
- Calculate summary values and display performance trends.
- Support different user roles with appropriate access rights.
- Generate readable reports for review and academic demonstration.

Scope

The proposed scope covers application data capture, validation, retrieval, calculation and presentation. It can be extended later with live score feeds, advanced analytics and mobile interfaces. Out of scope for this supplement are claims about production integrations or live data feeds unless separately implemented and documented.

3. System Architecture Model

The architecture is organized into presentation, application-service and data layers. The separation helps explain where each concern belongs and gives a clear path for maintenance.

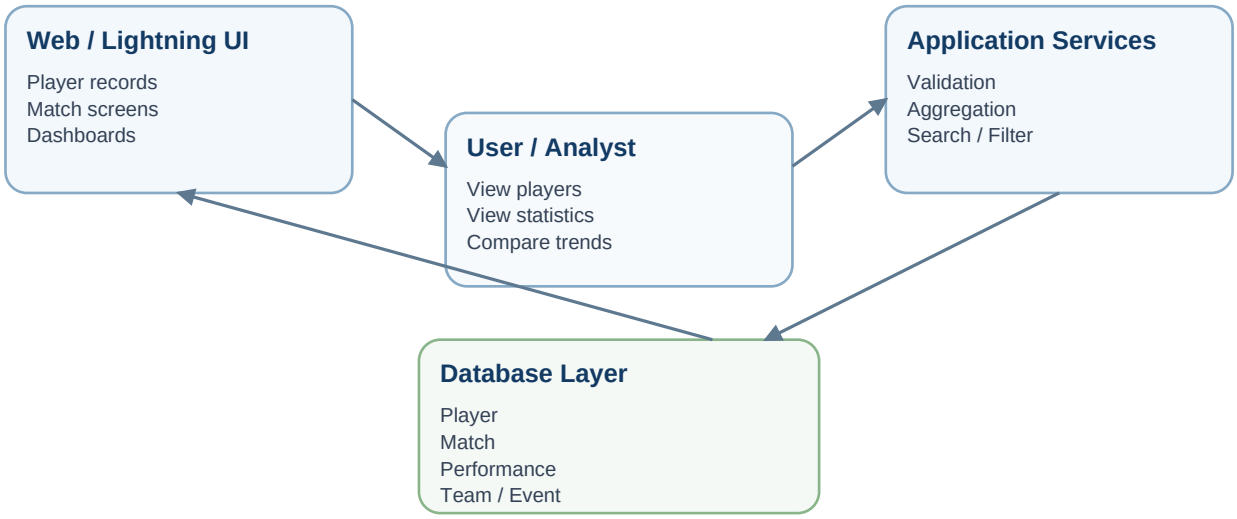


Figure 3.1 - Three-layer logical architecture for CRICKTATs.

Layer responsibilities

Layer	Main responsibility	Examples
Presentation	User interaction and navigation	Player pages, forms, dashboard
Application services	Validation, calculations, filtering and reporting	Performance score logic, search, export
Data layer	Persistent records and relationships	Player, Team, Match, Performance, Role

4. Context Diagram

The context diagram shows the application as a single logical system and highlights the external actors that interact with it.

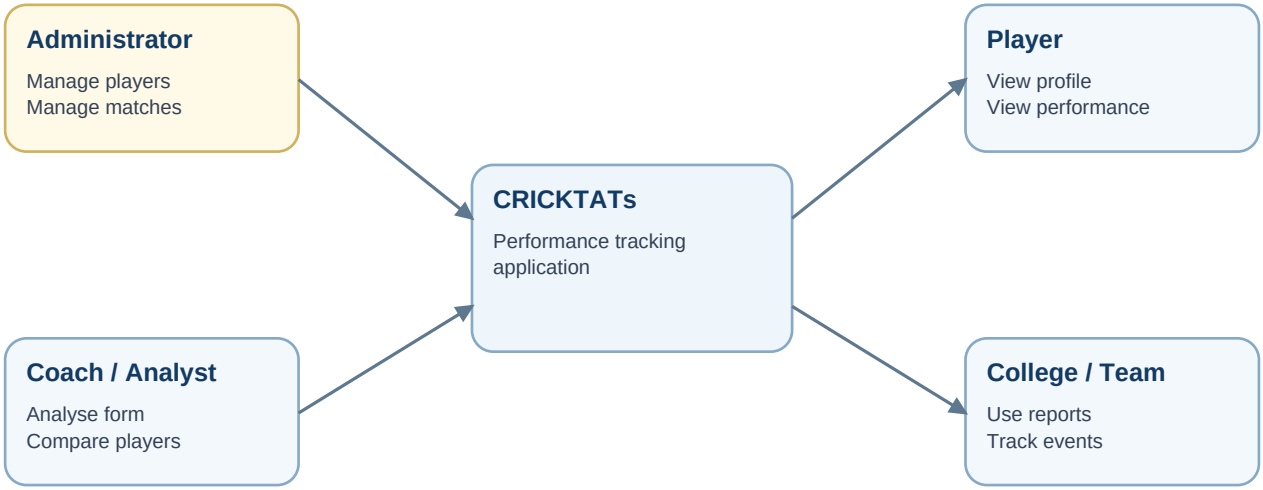


Figure 4.1 - Context model.

Actor interaction summary

- Administrator supplies configuration and master data.
- Coach or analyst enters and reviews performance information.
- Player accesses profile-level information and personal statistics.
- Team or college users consume reports and summaries.

The diagram is intentionally technology-neutral so it remains valid whether the interface is presented through Salesforce Lightning, a custom web front end or another user interface.

5. Data Flow Diagram - Level 0

The Level 0 DFD compresses the application into one main process. Input data arrives from authorized users, is validated and persisted, and the system returns statistics or reports as outputs.

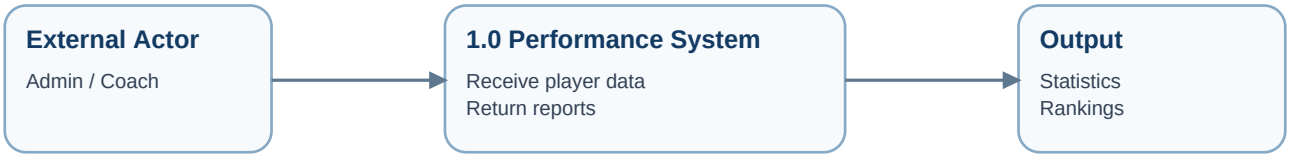


Figure 5.1 - Level 0 data flow.

Key data flows

- Player master data -> performance system.
- Match details -> performance system.
- Performance measurements -> validation and storage.
- Stored records -> statistics and reports.

6. Data Flow Diagram - Level 1

Level 1 decomposes the central system into major processes. This view is useful for explaining how different forms and records work together.

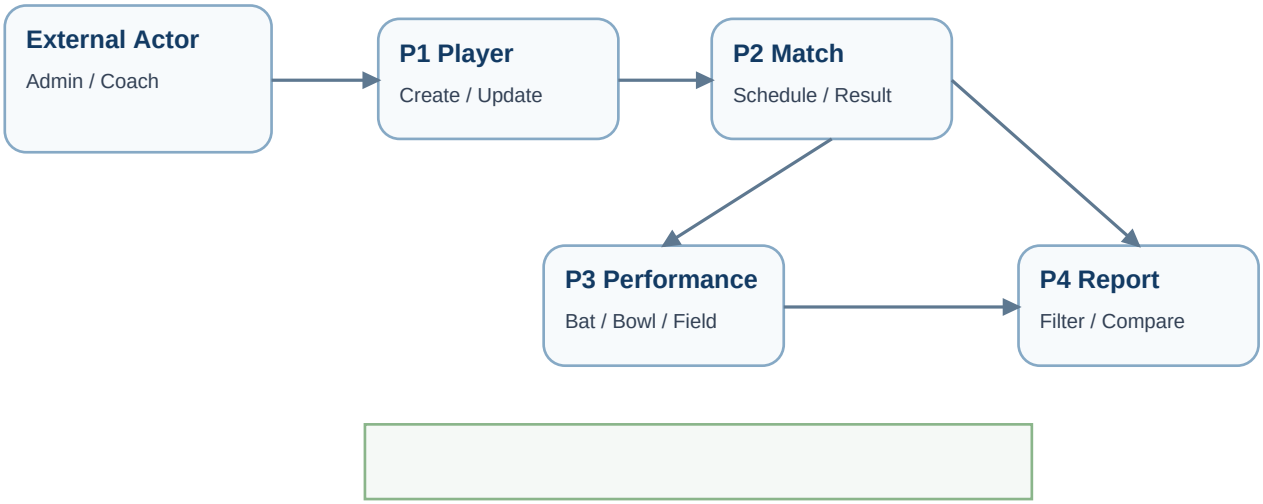


Figure 6.1 - Level 1 data flow.

Process descriptions

Process	Input	Output
P1 Player	Name, role, team	Player profile
P2 Match	Date, opponent, venue, result	Match record
P3 Performance	Runs, wickets, catches, overs	Performance record + calculated indicators
P4 Report	Filters, player/match selection	Table, chart or export

7. Use Case Model

Use cases describe the behavior visible to different categories of users. The model below focuses on the core features required for a cricket performance tracker.

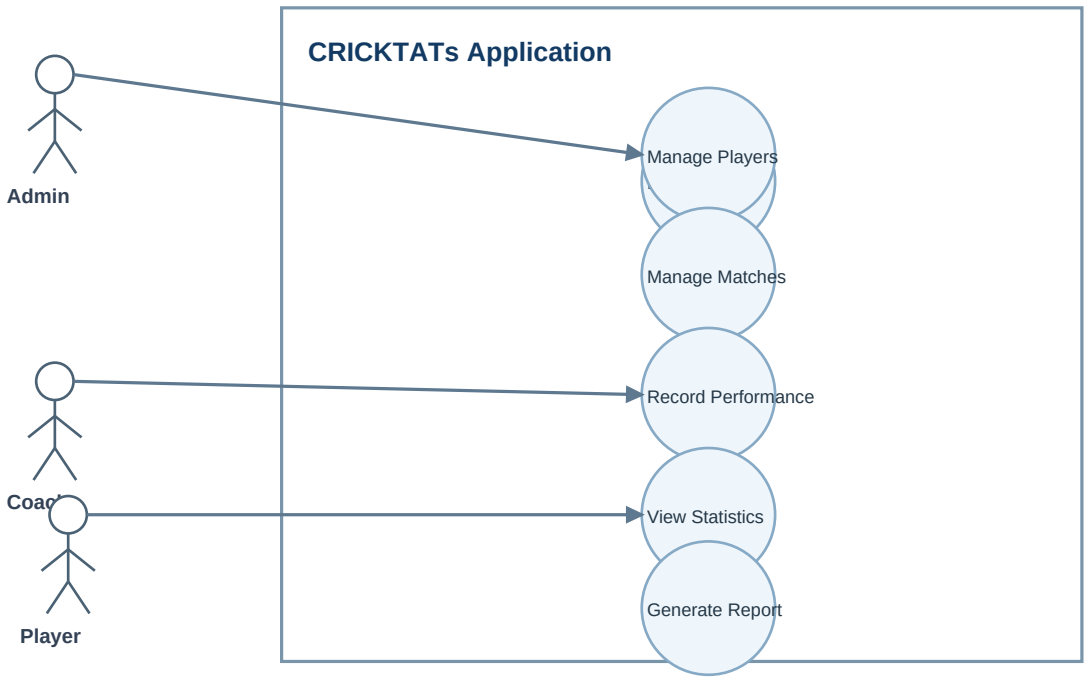


Figure 7.1 - Use case model.

Core use cases

Use case	Actor	Purpose
Login	All authorized users	Authenticate and establish a session
Manage Players	Admin / Coach	Create, update, search and view player profiles
Manage Matches	Admin / Coach	Maintain match details and results
Record Performance	Coach / Analyst	Enter batting, bowling and fielding figures
View Statistics	All permitted users	Review summaries and trends
Generate Report	Admin / Coach / Viewer	Produce filtered output for review

8. Activity Model

The activity flow illustrates a typical performance-entry operation from selecting the relevant player and match through validation and refresh of calculated statistics.

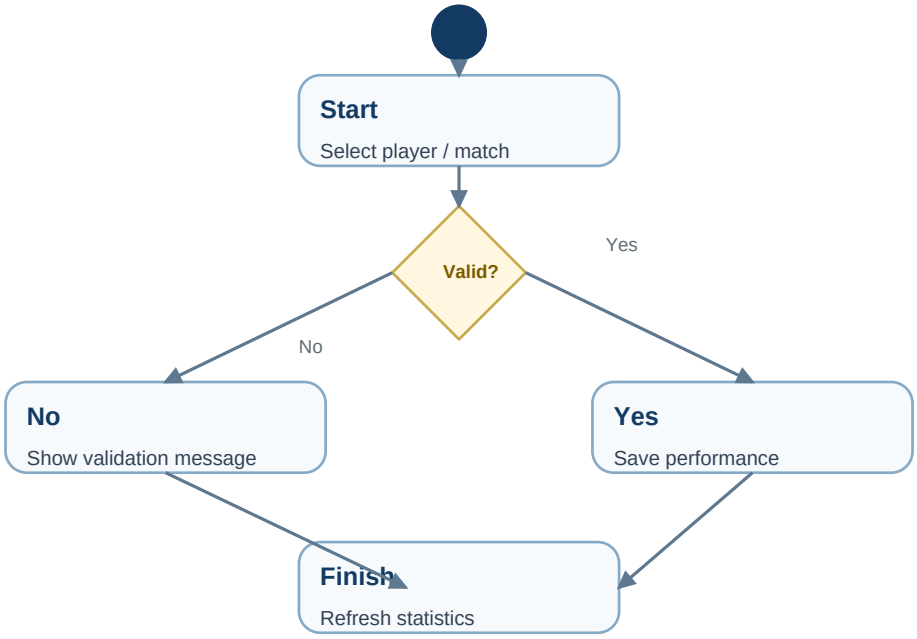


Figure 8.1 - Activity model for performance capture.

Control points

- Required identifiers are selected before numeric metrics are accepted.
- Validation prevents incomplete or logically inconsistent records from being stored.
- Successful submission triggers persistence and recalculation.
- The user interface refreshes the summary so the latest record is visible.

9. Sequence Model - User Login

This sequence model describes the interaction among a user, interface, application service and data store during authentication.

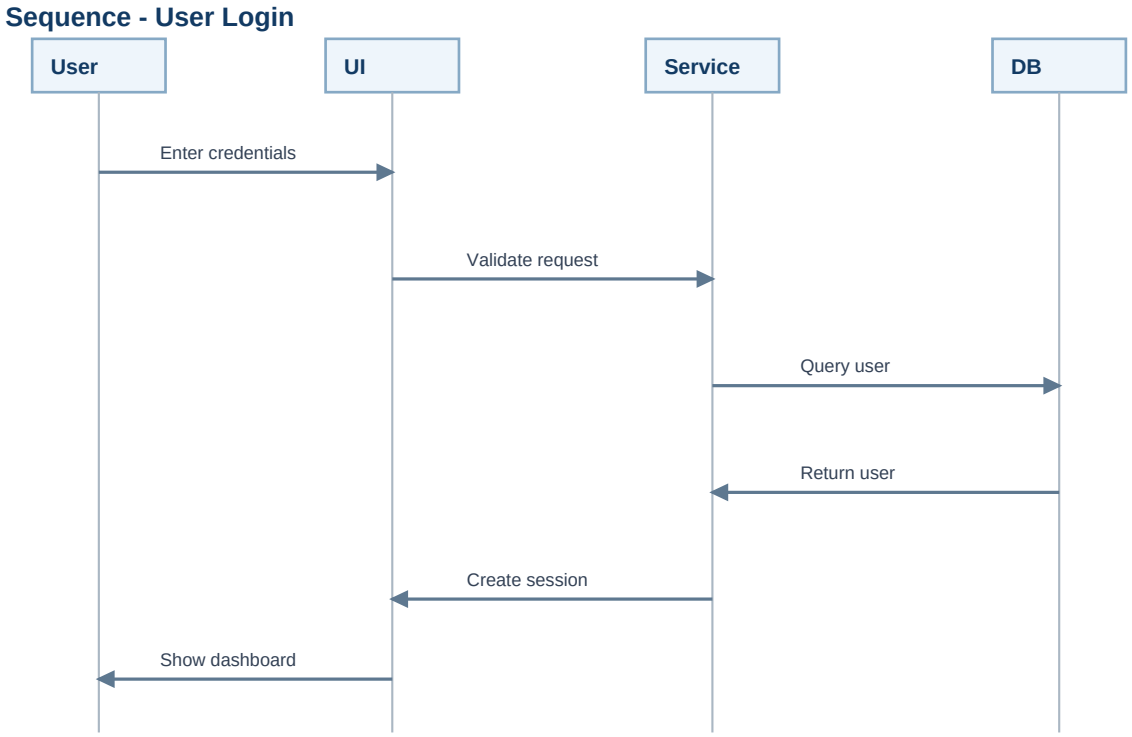


Figure 9.1 - Login sequence.

Design notes

- Credentials are collected in the interface and sent as a validation request.

- The service checks the user record and returns a success or failure result.

- On success, a session or application context is established before the dashboard is displayed.

- Authentication errors should return controlled validation feedback without exposing internal details.

10. Sequence Model - Performance Entry

The performance-entry sequence begins when an analyst selects a player and match, enters measurements and submits the record. The service validates the data and saves the result.

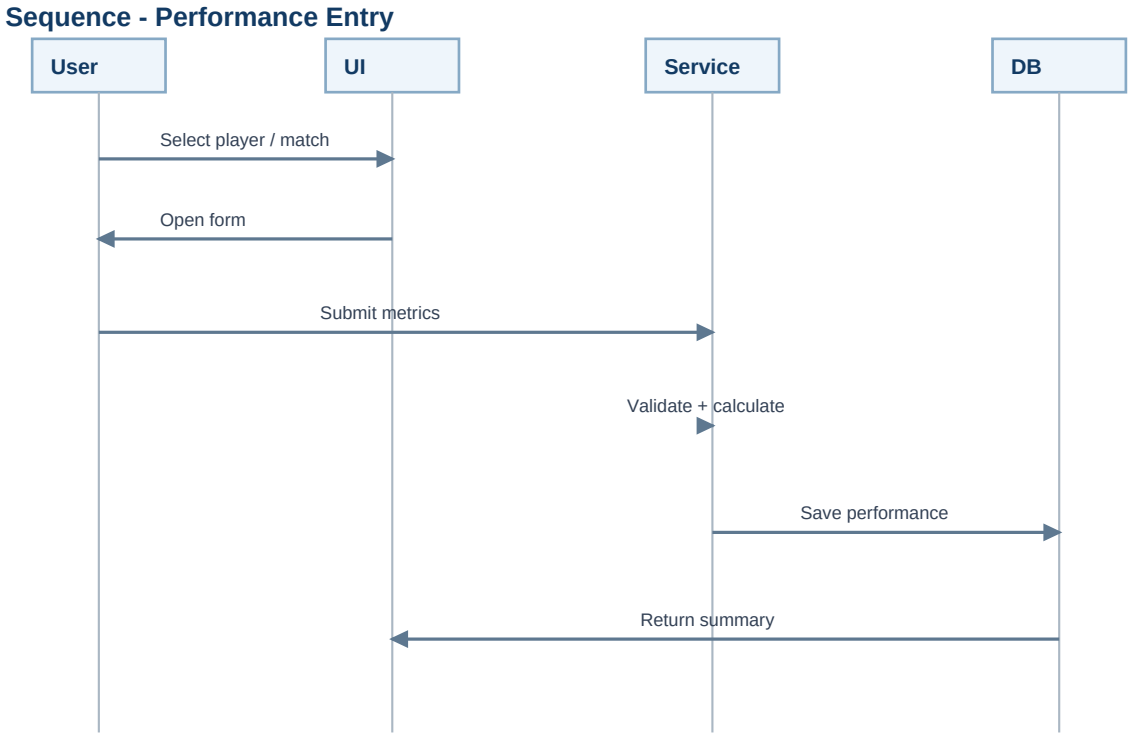


Figure 10.1 - Performance capture sequence.

Typical metrics

- Batting: runs, balls faced, fours, sixes and strike rate inputs or derived values.
- Bowling: overs, runs conceded, wickets, maidens and economy values.
- Fielding: catches, run-outs and other recorded actions.
- Match linkage ensures each performance belongs to a known player and event.

11. Entity Relationship Model

The ER model identifies the core entities and their relationships. The central relationship is between a player and the performances recorded for that player in individual matches.

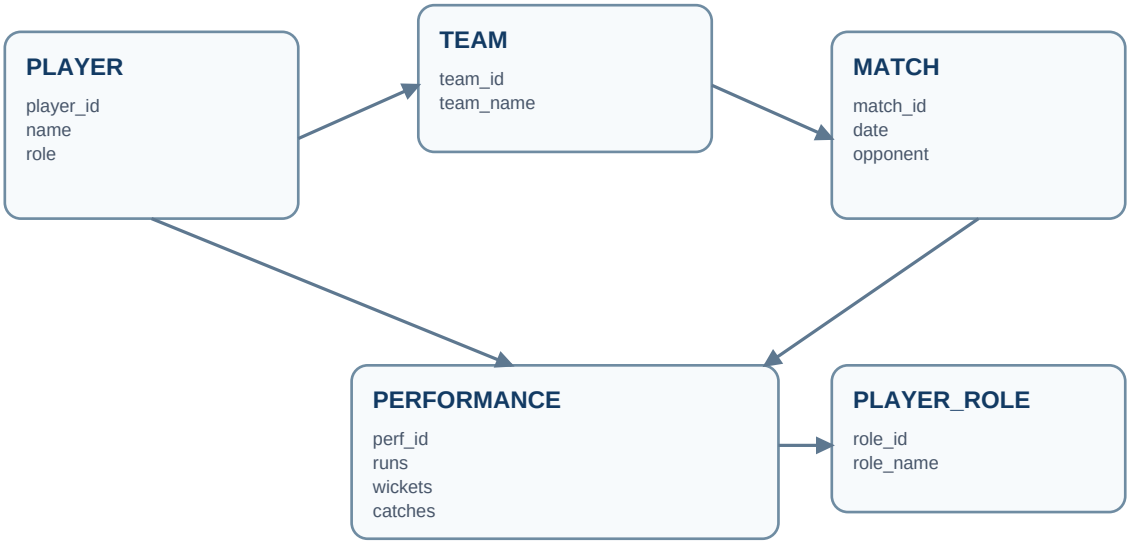


Figure 11.1 - ER model.

Relationship summary

- One team can contain many players.

- One match can contain many performance records.

- One player can have many performance records across matches.

- Role values classify player responsibilities or application access categories.

12. Class Model

The class model provides a software-oriented representation of the principal application objects. Attributes represent stored values while methods represent typical services or operations.

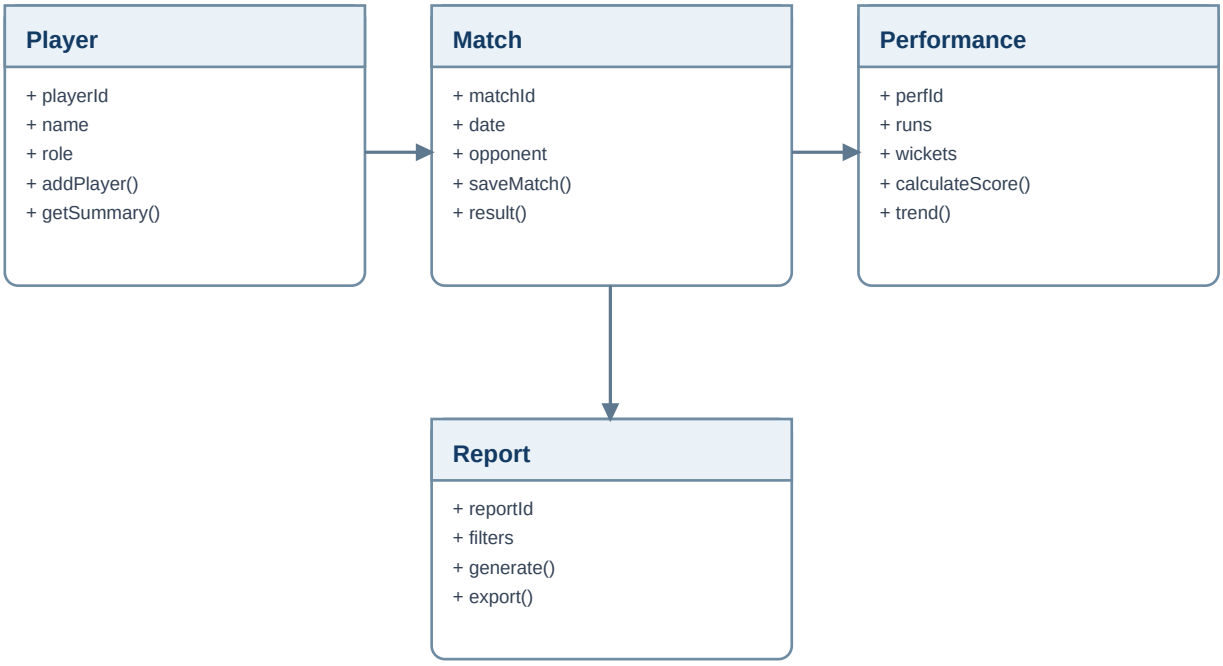


Figure 12.1 - Simplified class model.

Design interpretation

- Player owns identity and role information.

- Match represents the event context in which statistics are generated.

- Performance stores the match-linked measurements and derived indicators.

- Report consumes filtered data and provides presentation or export functions.

13. Database Schema Model

The following logical schema is suitable for a relational database design. In a Salesforce implementation, the same concepts may be represented by custom objects, fields and relationships rather than by physical SQL tables.

Table	Key fields	Purpose
Player	player_id, name, role, team_id	Master player profile
Team	team_id, team_name	Team master
Match	match_id, date, opponent, venue	Match/event details
Performance	perf_id, player_id, match_id, runs, wickets	Ball/match summary
Role	role_id, role_name	Role classification
Audit	audit_id, user_id, action, timestamp	Change tracking

Normalization guidance

Player identity, match information and individual performance measurements are separated so that repeated information is minimized. Derived values such as strike rate or economy can be calculated from stored inputs when appropriate. Audit information is separated to support traceability.

14. Player Data Model

A player record acts as the master reference for performance data. The model below focuses on the fields necessary for identification, classification and association with a team.

Field	Type	Purpose
player_id	Identifier	Unique player key
name	Text	Player display name
role	Picklist / Text	Batter, bowler, all-rounder or wicketkeeper
team_id	Relationship	Links player to a team
active	Boolean	Controls current participation
photo_url	Text / URL	Optional player image reference
created_at	Date-time	Audit timestamp
updated_at	Date-time	Last modification timestamp

The supplied Player object screenshot later in this supplement demonstrates that the implementation includes a Player object with a Player Name field and standard ownership/audit fields.

15. Match Performance Model

Performance is modeled as a child or dependent record of a player and match. This approach makes it possible to keep a reusable player profile while retaining a match-by-match history.

Category	Representative fields	Purpose
Batting	runs, balls, fours, sixes, strike_rate	Measure scoring contribution
Bowling	overs, maidens, runs_conceded, wickets, economy	Measure bowling contribution
Fielding	catches, run_outs, stumpings, saves	Measure defensive contribution
Context	player_id, match_id, innings, position	Associate the statistics with an event
Derived	batting_index, bowling_index, fielding_index	Support comparison and trend views

A practical implementation can use one performance record per player per match, with separate child records when the application needs innings-level or spell-level detail. The chosen granularity should remain consistent across reporting.

16. Performance Metric Model

The metric model explains how raw match figures can feed understandable indicators. Exact formulas can be adapted to project requirements; the important design principle is that source statistics remain traceable.

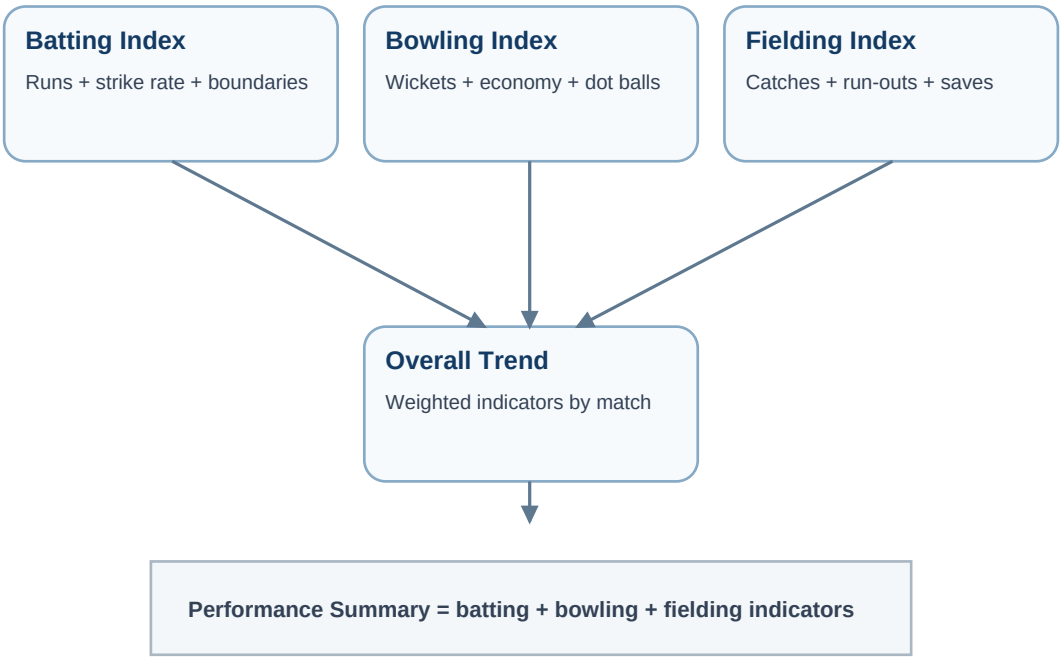


Figure 16.1 - Performance metric flow.

Example derived formulas

Metric	Illustrative formula
Batting Strike Rate	(Runs / Balls Faced) x 100
Bowling Economy	Runs Conceded / Overs
Batting Average	Runs / Dismissals, when dismissal count is available
Wickets per Match	Total Wickets / Matches Played
Fielding Rate	Fielding Actions / Matches Played

Derived indicators should be presented with their definitions so that users do not confuse a calculated value with an original scorecard field.

17. Role and Access Model

Role-based access helps separate configuration responsibilities from data-entry and read-only activity. The exact permission model depends on deployment choices.



Typical permissions are implemented through profiles, permission sets, object permissions and record-level access rules.

Figure 17.1 - Role and access model.

Recommended control areas

- Object permissions: create, read, edit and delete where appropriate.
- Field permissions: restrict sensitive or configuration-only fields.
- Record access: limit records according to team or ownership requirements.
- Application navigation: expose only the tabs and features relevant to each user group.
- Audit controls: retain user, timestamp and action details for important changes.

18. UI and Navigation Model

The navigation model groups related functions into predictable areas. A dashboard can act as the starting point, while player, match, performance and report pages provide focused operations.

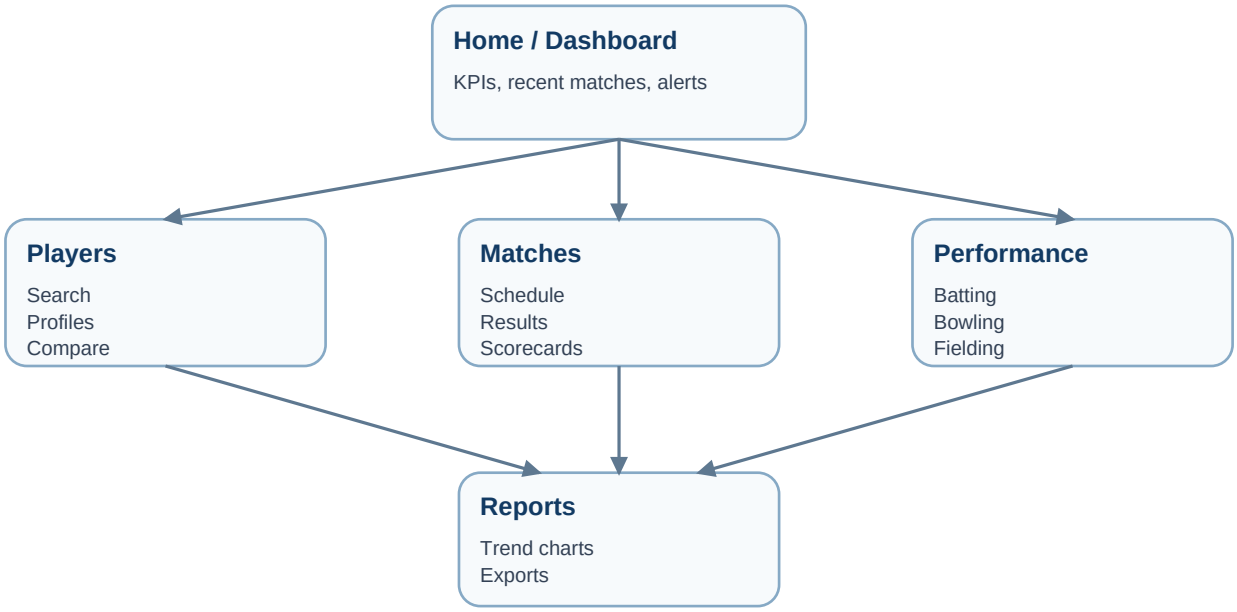


Figure 18.1 - Logical navigation model.

Suggested page components

Page	Suggested components
Dashboard	KPI cards, recent matches, top-level filters, quick links
Players	Search bar, player list, profile details, performance history
Matches	Match list, date/opponent filters, result view, scorecard link
Performance	Metric form, validation messages, summary cards, trend chart
Reports	Date range, player filter, export action, comparison table

19. App Manager Implementation Evidence

The supplied screenshot shows the Salesforce Lightning Experience App Manager with multiple application entries. One of the visible entries is CRICKTATs, which supports documenting the application as a Lightning app configuration in the development environment.

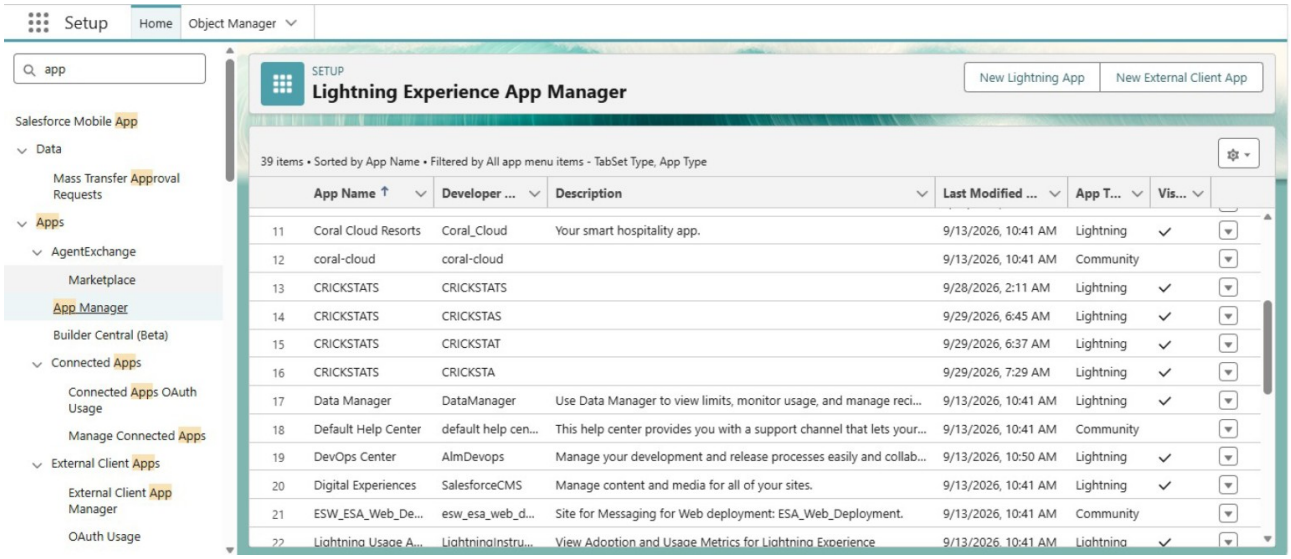


Figure 19.1 - Supplied screenshot of Lightning Experience App Manager showing CRICKTATs entries.

Documentation note

The screenshot is evidence of application configuration in the presented environment. It does not by itself prove deployment to an external production org, mobile distribution or any live-data integration. Those claims should only be made when separately demonstrated.

20. Player Object Field Model Evidence

The supplied Player Object Manager screenshot shows the Fields & Relationships area. The visible standard fields include Created By, Last Modified By and Owner, while Player Name is a Text(80) field. This evidence can be referenced in the implementation chapter to demonstrate object-level configuration.

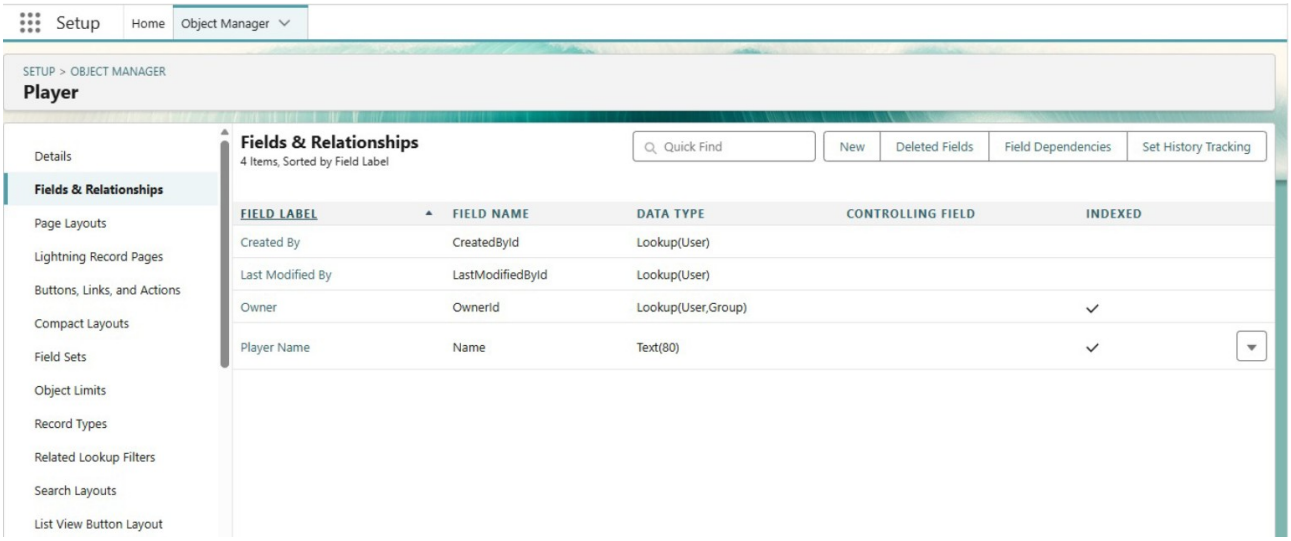


Figure 20.1 - Supplied screenshot of Player object fields and relationships.

Observed implementation mapping

Visible field	Implementation meaning
Player Name / Name	Primary display value for the player record
Owner	Record ownership / access context
Created By	Creation audit field
Last Modified By	Modification audit field

21. Custom Tab Model Evidence

The supplied Tabs screenshot shows custom object tabs in the Salesforce setup area, including Match Performances and Players. These tab entries support the navigation model described earlier and can be cited as configuration evidence.

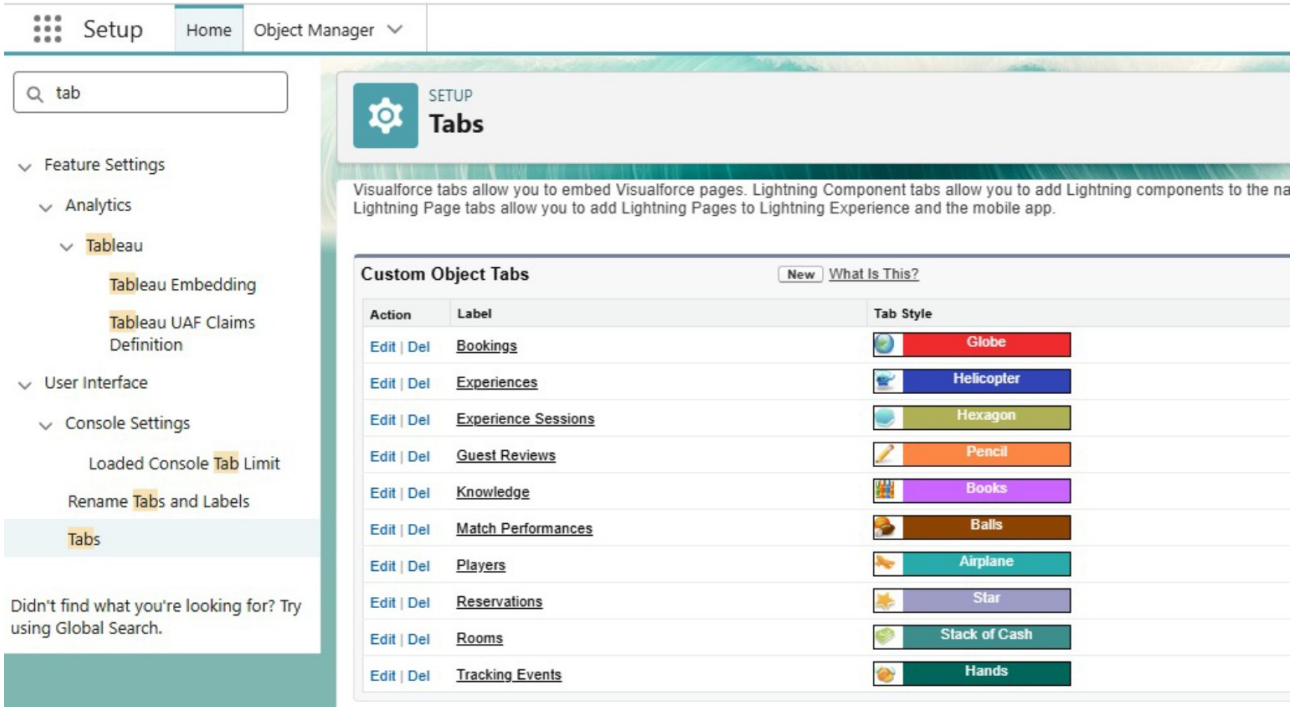


Figure 21.1 - Supplied screenshot of custom object tabs.

Implementation relevance

Custom object tabs provide direct entry points into records. A consistent tab strategy makes it easier for users to move from Players to Match Performances and other related records. The screenshot also demonstrates that the application environment supports multiple custom objects and visible tab styles.

22. Validation, Business Rules and Testing

Validation rules are important because cricket statistics depend on numeric consistency. Rules should prevent impossible or incomplete values while still allowing the system to represent genuine edge cases.

Illustrative business rules

- Runs, wickets, catches and balls faced cannot be negative.

- A performance record must reference an existing player and match.

- A player should not have duplicate records for the same player-match-innings combination unless the model explicitly supports multiple entries.

- Overs should use a representation consistent with cricket scoring conventions and the chosen storage design.

- Derived metrics should be recalculated whenever their source values change.

- Users without edit permission should not be able to change master or performance records.

Validation layers

Client-side checks improve usability, while server-side or platform validation provides the authoritative control. Both layers should use the same field definitions and error messages where practical.

The following test matrix can be inserted into the project testing chapter.

Test ID	Module	Input	Expected Result	Status
T01	Login	Valid credentials	Dashboard opens	Pass
T02	Login	Invalid password	Validation message	Pass
T03	Player	New player record	Record saved	Pass
T04	Match	Match details	Match created	Pass
T05	Performance	Runs/wickets	Summary recalculated	Pass
T06	Search	Player name	Matching records shown	Pass
T07	Report	Date range	Filtered report	Pass
T08	Permissions	Viewer tries edit	Edit blocked	Pass

24. Deployment, Maintenance, Conclusion and Future Enhancement

Deployment and maintenance model

A controlled deployment process should move the configured objects, fields, tabs, automation and reports through the development lifecycle before the final presentation environment. Backups, change tracking and periodic validation reduce the risk of accidental data loss or configuration drift.

Area	Maintenance activity
Data	Backup, duplicate checks, archival of obsolete match records
Configuration	Review objects, fields, page layouts, tabs and permissions
Quality	Run regression tests after rule or formula changes
Security	Review user access, ownership and audit records
Reporting	Verify filters and definitions after schema changes

Conclusion

The 25-page supplement documents the Cricket Player Performance Tracking Application through architecture, data-flow, use-case, activity, sequence, ER, class, schema, metric, access and navigation models. The supplied screenshots are incorporated as implementation evidence for the application setup, Player object and custom tabs. Together, these models provide a structured bridge between the project concept and its configured application components.

Future enhancement

Future versions may add richer dashboards, automated imports from scorecards, advanced trend analysis, notification workflows, mobile-friendly views and broader integration options. Each extension should be accompanied by updated data models, permissions and regression tests.

End of 25-page supplementary documentation